ARABIAN SOCIAL NETWORK · REFERENCE DOCUMENT

Chat Contract Slice One

REST contract, event schemas, and the protocol-neutral `chat_db` model

The first Chat slice defines **what** the service exposes and **how** state is stored before any endpoint is implemented. It is deliberately protocol-neutral: no MLS columns, no KeyPackage tables, no plaintext. Development envelopes use `dev-opaque-v1`, which has the same shape as production ciphertext.

OPENAPI OPERATIONS	EVENT PAYLOAD SCHEMAS	MIGRATIONS	TABLES CREATED
10	**4**	**7**	**7**

CONTRACT VERSION	OPENAPI	DATABASE	BRANCH
1.0.0	**3.1.0**	**PostgreSQL 18.6**	**feat/chat-contract-slice-one**

Contract-first order. Endpoints are not implemented in this slice. The contract pins the surface — paths, status codes, idempotency, pagination, error codes — so implementation, clients, and tests converge on one shape. Source of truth stays in `contracts/openapi/chat-api.yaml`, `contracts/events/chat/`, and `docs/architecture/architecture-and-schema.md`; this document is a read-only summary of those files.

Contents

1	Executive summary and scope	2
2	REST API surface	3
3	Conventions: errors, idempotency, pagination, payloads	4
4	Event contracts and the envelope	5
5	Data model	6
6	Integrity rules and constraints	8
7	End-to-end workflows	9
8	Design decisions and rationale	10
9	Verification	11
10	Open risks and follow-ups	12
11	References	13

1 Executive summary and scope

10
REST OPERATIONS
Devices, conversations, membership, messages, and read cursor — every operation carries stable error codes.

0
PLAINTEXT FIELDS
No `body` column, DTO, event field, log line, or fixture carries decrypted content.

0
PROTOCOL COMMITMENTS
No `device_key_packages`, no `messages.crypto_epoch`, no MLS ciphersuite columns.

In this slice INCLUDED

- OpenAPI contract for the first vertical slice (send, read, resync).
- Four chat event payload schemas plus envelope fixtures.
- Seven neutral `chat_db` migrations with constraints verified against PostgreSQL 18.6.
- Swagger UI viewer, READMEs, and architecture catalog rows.

Deferred EXCLUDED

- Controllers, contexts, routes, and channels — no endpoint is implemented yet.
- Attachments, message receipts, reactions, edits, and deletions.
- Prekeys, KeyPackages, control messages, and commit authorization (ADR 0014 is still *proposed*).
- Friendship, privacy, and block projections — owned by a later Social-integration slice.

Why contract first

The repository rule is explicit: API and event changes update `contracts/` before code. Pinning the surface first keeps the design reviewable, keeps mobile and web clients unblocked, and makes the later implementation diff a verification of an agreed shape rather than a new proposal.

2 REST API surface

Base path `/api/v1`, plural resources, bearer security (provisional pending architecture §9.1). Every conversation-scoped read answers `404` for a non-member so the existence of a conversation never leaks.

#	OPERATION	PATH	IDEMPOTENCY	KEY RESPONSES
1	`registerDevice`	`POST /devices`	Same active fingerprint + identical key → `200`; same fingerprint + different key → `409 identity_key_conflict`	`201 200 409 413 422`
2	`listDevices`	`GET /devices`	Read only; caller's own devices, bounded set	`200`
3	`revokeDevice`	`POST /devices/{deviceId}/revoke`	Repeat returns current state, never `409`	`200 404`
4	`createConversation`	`POST /conversations`	Deterministic `direct_key`; re-create → `200` with the existing conversation	`201 200 403 422`
5	`listConversations`	`GET /conversations`	Cursor over (`last_message_at DESC NULLS LAST, id DESC`)	`200 422`
6	`getConversation`	`GET /conversations/{conversationId}`	Read only	`200 404`
7	`listConversationMembers`	`GET /conversations/{conversationId}/members`	Active members only; cursor paged	`200 404 422`
8	`sendMessage`	`POST /conversations/{conversationId}/messages`	`client_message_id`: identical decoded body → `200`; any difference → `409 duplicate_client_message_id`	`201 200 409 404 413 422`
9	`listMessages`	`GET /conversations/{conversationId}/messages`	Exactly one of `cursor`, `after_seq`, `before_seq`	`200 400 404 422`
10	`updateReadCursor`	`POST /conversations/{conversationId}/read-cursor`	Forward-only: a lower value is a no-op that echoes the stored value	`200 404 422`

CORE RESPONSE SCHEMAS

`ConversationView`

`conversation_id`, `kind`, `title?`, `community_id?`, `channel_id?`, `last_seq`, `last_message_at?`, `crypto_protocol`, `crypto_protocol_version`, `created_at`, and a required **`viewer`** object: `role`, `last_read_seq`, `joined_at`, `muted_until?`, `membership_version`.

`MessageView`

`message_id`, `conversation_id`, `seq`, `sender_id`, `sender_device_id`, `client_message_id`, `payload_kind`, `payload_mode`, `reply_to_id?`, `revision_no`, `payload?` (latest revision; absent only when `deleted_at` is set), `edited_at?`, `deleted_at?`, `created_at`, `updated_at`.

`DeviceView`

`device_id`, `platform`, `device_label?`, `identity_key_public` (base64), `identity_key_fingerprint`, `status`, `created_at`, `last_seen_at?`, `revoked_at?`. The public key is returned over REST for verification and never appears in an event.

`MemberView`

`user_id`, `role`, `joined_at`, `membership_version`. Another member's `last_read_seq` is never exposed: read state is private to its owner.

3 Conventions: errors, idempotency, pagination, payloads

Error model — RFC 9457 Problem Details

Content type `application/problem+json` with `type`, `title`, `status`, `detail?`, a stable `code`, and `request_id` for correlation.

Stable codes: `unauthorized`, `forbidden`, `forbidden_direct_chat_not_permitted`, `not_found`, `conflict`, `invalid_request`, `unprocessable`, `payload_too_large`, `rate_limited`, `identity_key_conflict`, `duplicate_client_message_id`, `service_unavailable`.

Opaque payload encoding

```
OpaquePayload:
  required: [payload_encoding, ciphertext]
  payload_encoding: { enum: [base64] }
  ciphertext:       { type: string, format: byte }
```

Base64 is the only v1 value and is never inferred: an absent `payload_encoding` is a validation error, not a default. The server decodes to `bytea` and stores ciphertext only. Request bodies are capped at **256 KiB**, the same figure the broker enforces, so one limit governs the platform.

IDEMPOTENCY RULES

OPERATION	KEY	SAME KEY, SAME CONTENT	SAME KEY, DIFFERENT CONTENT
`sendMessage`	`client_message_id` (unique per sender in `messages`)	`200` + original `MessageView`	`409 duplicate_client_message_id`
`registerDevice`	`(user_id, identity_key_fingerprint)` while active	`200` + existing device	`409 identity_key_conflict`
`createConversation`	Deterministic `direct_key` over the sorted pair	`200` + existing conversation	n/a (a direct key has one shape)
`revokeDevice`	Device state	`200` + current state	n/a
`updateReadCursor`	Forward-only monotonic cursor	Lower value → no-op `200` echoing stored value	> `last_seq` → `422`

Comparison is on decoded values

A retry that reorders JSON keys is the *same* request. Duplicate detection compares `reply_to_id` and the decoded `payload` bytes within the same conversation — never raw request bytes. This is documented in the contract so an implementation cannot invent its own interpretation.

PAGINATION

- Opaque `cursor` tokens (base64url) are issued per resource and bound to their path; a foreign token answers `422`.
- `listMessages` accepts exactly one of `cursor`, `after_seq` (reconnect recovery), or `before_seq`; more than one answers `400`.
- Limits: messages 1–200 (default 50), inbox 1–100 (default 20), members 1–200 (default 50). Offset pagination is not supported anywhere.
- Message ordering is `seq` ascending; inbox ordering is `(last_message_at DESC NULLS LAST, id DESC)`. `last_message_at` is inbox metadata, never a sync cursor.

4 Event contracts and the envelope

Payload schemas live in `contracts/events/chat/`. Each file is a JSON Schema draft 2020-12 that constrains the `payload` object; the shared envelope in `contracts/events/common/` constrains everything else and enforces all-or-none aggregate fields.

EVENT TYPE	PAYLOAD FIELDS	ENVELOPE CONTRACT
`chat.device.registered`	`device_id`, `user_id`, `platform`, `identity_key_fingerprint`	`aggregate_type=device`, `aggregate_id=device_id`, `aggregate_version=1`
`chat.device.revoked`	`device_id`, `user_id`	`aggregate_type=device`, `aggregate_id=device_id`, `aggregate_version=2`
`chat.conversation.created`	`conversation_id`, `kind=direct`, `member_user_ids[2]`, `crypto_protocol`, `crypto_protocol_version`	No aggregate fields — creation is a one-shot fact, not a versioned mutation
`chat.message.created`	`conversation_id`, `message_id`, `seq`, `sender_id`, `sender_device_id`, `payload_kind`, `payload_mode`, `reply_to_id`	`aggregate_type=message`, `aggregate_id=message_id`, `aggregate_version=1`

Never in an event

- Ciphertext or any payload bytes — recipients resync over REST.
- Raw public key material — REST-only, in `DeviceView`.
- `device_label` — untrusted user text with no consumer.
- Read cursors — private to their owner.

Aggregate choice for messages

Messages use `aggregate_type=message` rather than `conversation` + `seq`. Overloading the conversation version space would collide with every future conversation-scoped event, and `crypto_state_version` is an epoch counter, not a general event counter. Per-conversation order is carried by `seq` inside the payload; broker ordering is never used as a conversation feed.

DELIVERY SEMANTICS

1. Business change and `outbox_events` row are written in one local transaction.
2. The relay publishes after commit, reusing a stable `event_id`; the broker answers `200` for an identical duplicate and `409` for the same `event_id` with different content.
3. Delivery is at-least-once. Consumers deduplicate on `event_id` into `processed_events` and reject stale `aggregate_version` values atomically with their business effect.
4. There is no global ordering guarantee — only per-aggregate ordering.

> **Catalog discipline**
>
> `chat.conversation.created` and `chat.message.created` were added to the architecture's event catalog (§7) in the same change, so the catalog and the payload schemas land atomically. Every event a service emits must be cataloged before its producer exists.

5 Data model

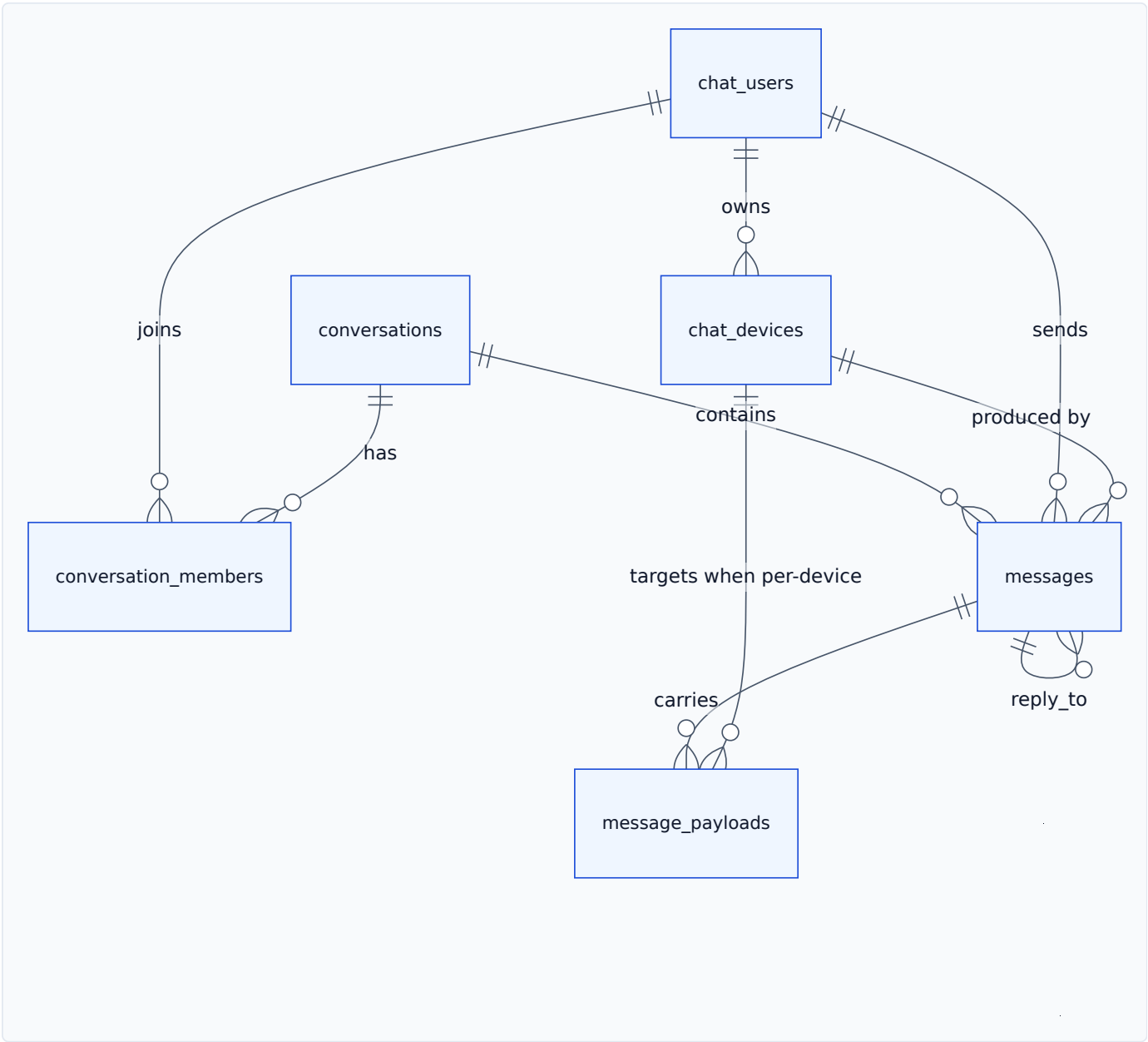


Figure 1 — `chat_db` entity relationships. All foreign keys stay inside `chat_db`; IDs that reference Social data (`community_id`, `channel_id`) are soft references, never foreign keys.

TABLE	PURPOSE	KEY CONSTRAINTS
`chat_users`	Read-only projection of `social_db.users`	Same UUID as Social; no unique on `username` — Social is the source of truth
`chat_devices`	One cryptographic identity per installed client	Partial unique `(user_id, identity_key_fingerprint) WHERE status='active'`; unique `(id, user_id)` backing a composite FK
`conversations`	Direct or community conversation and its ordering head	`direct_key` unique; `channel_id` unique; kind-guarded shape CHECKs; `last_seq` default 0
`conversation_members`	Membership, role, and the member's read cursor	Composite PK `(conversation_id, user_id)`; role CHECK; partial index for active membership
`messages`	Logical message and routing metadata — no content column	Unique `(conversation_id, seq)`, `(sender_id, client_message_id)`, `(id, conversation_id)`
`message_payloads`	Opaque ciphertext, one row per revision and target	Partial uniques for shared and per-device envelopes; `ciphertext bytea NOT NULL`
`processed_events`	Consumer-side deduplication ledger	PK `event_id`; index on `(aggregate_type, aggregate_id)` for stale-version checks

What is deliberately absent

No `body` column, no `crypto_epoch`, no `device_key_packages`, no ciphersuite columns, no partitioning, and no server-side extension of `outbox_events` beyond what the existing migration already defines. UUIDs are minted in application code (UUIDv7), never by the database, so migrations contain no `gen_random_uuid()`.

6 Integrity rules and constraints

Sequence issuance is the only serialization point

```
-- one transaction, one row lock
UPDATE conversations
   SET last_seq = last_seq + 1
 WHERE id = $1
RETURNING last_seq;
```

Messages in different conversations proceed concurrently; messages in the same conversation serialize on this row. No global writer exists anywhere in the design.

Composite foreign keys that pin identity

```
messages(sender_device_id, sender_id)
  → chat_devices(id, user_id)

messages(reply_to_id, conversation_id)
  → messages(id, conversation_id)
```

A device can never appear to belong to another sender, and a reply can never point at a message in a different conversation — enforced by the database, not by application code.

Envelope shape is a partial unique, not a CHECK

```
UNIQUE (message_id, revision_no)
  WHERE recipient_device_id IS NULL

UNIQUE (message_id, revision_no,
recipient_device_id)
  WHERE recipient_device_id IS NOT NULL
```

Shared and per-device rows for the same revision are mutually exclusive at the index level. Mixing them is a service-layer invariant — a row-level CHECK cannot prove the absence of sibling rows.

Enums kept in sync with the audit table

`conversations.kind` → `direct`, `community`
`conversation_members.role` → `member`, `admin`
`chat_devices.status` → `active`, `revoked`, `pending_verification`
`chat_devices.platform` → `android`, `ios`, `web`
`messages.payload_kind` → `text`, `media`, `voice`, `system`, `control`
`messages.payload_mode` → `shared`, `per_device`

`payload_kind` keeps `control` because the audit table already enumerates it; the slice's contract still fixes sends to `text` + `shared`.

Kind-guarded shape rules

`conversations` carries two guarded CHECKs — direct rows must set `direct_key` and clear both community columns; community rows must do the reverse. Each rule is guarded by `kind` so it constrains only its own kind, while `kind` itself is constrained separately. The architecture text was corrected in this change to describe exactly that form.

7 End-to-end workflows

7.1 SENDING A MESSAGE — THE CRITICAL PATH

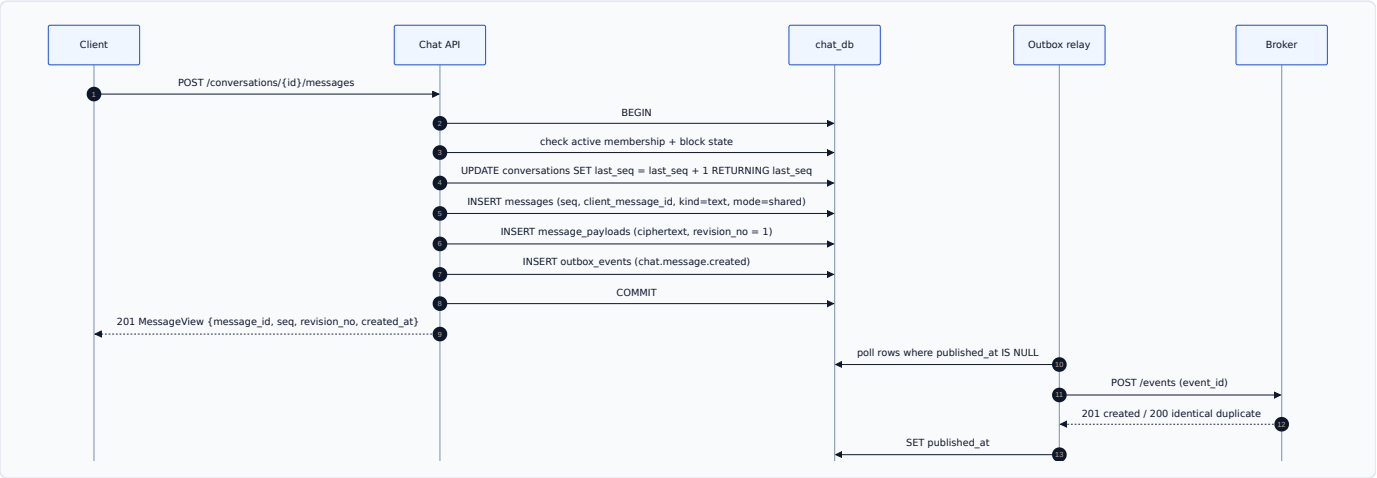


Figure 2 — Acknowledgement happens only after commit; the event is written in the same transaction as the business change, and publication happens after commit.

> **Retry behaviour on this path**
>
> A retry with the same `client_message_id` and the same decoded body returns `200` with the stored message; a concurrent race resolves against the unique index and re-reads the winner rather than returning a 500. A different body under the same id answers `409`.

7.2 RECOVERY AND RESYNC

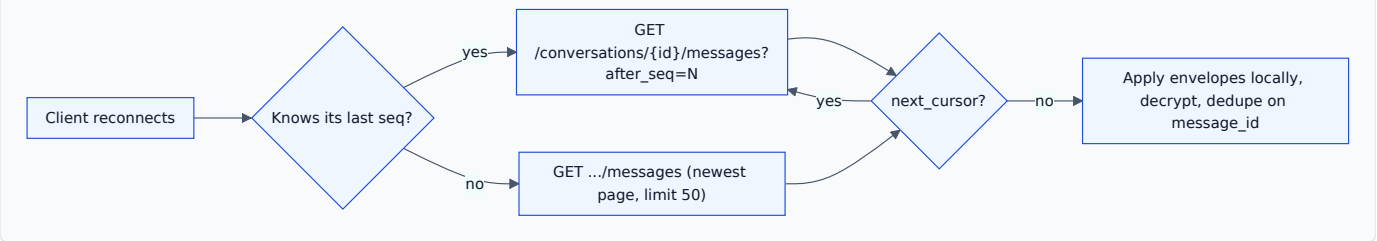


Figure 3 — Durable recovery always goes through REST with a stable `seq` cursor. Live delivery may be missed; history never depends on it.

7.3 DEVICE LIFECYCLE AND CONVERSATION CREATION

Register / revoke a device

1. `POST /devices` with `identity_key_public` + fingerprint (caller from auth, never from body).
2. Same active fingerprint and identical key → `200`; different key → `409 identity_key_conflict`.
3. Insert `chat_devices` and `chat.device.registered` (aggregate `device`, version 1) in one transaction.
4. `POST /devices/{id}/revoke` sets `status='revoked'` and emits version 2; repeats stay `200`.

Create a direct conversation

1. `POST /conversations {peer_user_id}`; the caller comes from auth — self-peer answers `400`.
2. Derive the deterministic `direct_key` from the sorted pair.
3. Insert `conversations` plus two `conversation_members` rows in one transaction; a repeat answers `200`.
4. Emit `chat.conversation.created` with no aggregate fields.
5. Block, friendship, and privacy checks answer `403 forbidden_direct_chat_not_permitted` once the projections land.

8 Design decisions and rationale

#	DECISION	RATIONALE
1	Messages use `aggregate_type=message`	Avoids exhausting the conversation version space that future receipts, tombstones, and membership events will need; order lives in `seq`.
2	`createConversation` takes `peer_user_id` only	The caller is authoritative from auth; sending your own id invites canonicalisation mistakes and pointless `400`s.
3	`ConversationView` embeds `viewer` state	Role and read cursor in the same response removes a second round trip and lets clients compute unread without inventing an endpoint.
4	Non-members receive `404`, not `403`	Indistinguishable from "does not exist" — guessing a UUID must not confirm a conversation exists.
5	`identity_key_public` is REST-only	Verification needs the key, but events stay metadata-only; a 256 KiB payload budget must never be consumed by key material.
6	Idempotency compares decoded values	JSON key order is not semantic; a client retry must never turn into a spurious `409`.
7	`payload` is optional in `MessageView`	A future tombstone returns metadata without ciphertext; requiring the field would force a breaking change at deletion time.
8	Single-value enums removed from the send request	Fields the caller cannot vary are noise; server-side defaults keep the request small and future values additive.
9	256 KiB body limit, shared with the broker	One platform-wide number instead of per-service limits that drift apart.
10	`payload_encoding: base64` is required	Binary envelopes cannot appear in JSON implicitly; explicit encoding forbids sniffing and silent defaults.
11	Shared `503` response on every operation	Generated clients need to know outage behaviour; previously only the broker contract documented it.
12	Protocol-neutral storage only	Accepting MLS now would commit to a library gate that has not closed; `dev-opaque-v1` keeps the contract stable across the protocol decision.

9 Verification

Every gate below was executed; outputs are reproduced verbatim.

```
$ python3 scripts/contracts/validate.py --base-ref origin/main
OpenAPI OK: contracts/openapi/chat-api.yaml (3.1.0, version 1.0.0)
OpenAPI OK: contracts/openapi/event-broker-api.yaml (3.1.0, version 1.0.0)
Event schemas OK: 5 schema(s), 6 valid and 4 invalid example(s)
Contracts validation OK.

$ python3 scripts/ci/check-links.py
Relative documentation links OK.

$ python3 scripts/ci/check-changelog.py --all
Change records OK.

$ cd services/chat-service
$ mix format --check-formatted            exit 0
$ mix compile --warnings-as-errors        exit 0
$ mix test
Running ExUnit with seed: 567515, max_cases: 24
Result: 6 passed
```

GATE	WHAT IT PROVES
`validate.py`	Both OpenAPI documents parse under 3.1, every `$ref` resolves, every payload schema is valid JSON Schema, all fixtures satisfy the envelope (6 valid / 4 invalid).
`check-links.py`	Every relative link added to the architecture, READMEs, and changelog resolves.
`check-changelog.py`	The change record exists with valid front matter and required sections.
`mix test`	The test alias creates and migrates `chat_db_test`, so all seven migrations ran against a real PostgreSQL 18.6.
`ecto.rollback --all` + `ecto.migrate`	Every migration is reversible; composite FKs, partial unique indexes, and CHECK constraints were re-verified through `pg_indexes` / `pg_constraint`.

Independent review

A separate read-only reviewer inspected the full diff against the architecture document, ADR 0007, ADR 0008, and the contract conventions. The verdict was *approve with fixes*: six should-fix findings (a doc/SQL wording mismatch, a missing `404`, versioning wording, a missing §8 enum row, `payload` optionality, and undefined duplicate comparison) plus six minor ones were all applied and re-verified before this document was generated.

10 Open risks and follow-ups

RISK	IMPACT	OWNER / NEXT STEP
Any future conversation-scoped event needs its own version counter	Choosing `aggregate_type=message` trades immediate ordering for headroom; a receipt or membership event cannot reuse `seq`.	An ADR introducing `conversations.event_version` before such an event ships. Not `crypto_state_version`.
`outbox_events` shape drifts from architecture §3.18	The migration carries `event_id` + `envelope`; §3.18 recommends `type`, `schema_version`, `payload`, `trace_id`.	Reconcile before the events context is implemented.
Validator does not bind payload schemas to event types	A payload schema can be wrong and still pass CI; fixtures are validated against the envelope only.	Tooling follow-up in `scripts/contracts/validate.py`.
Authentication strategy is provisional	Scheme name and `401`/`403` semantics are stable; the credential format is not.	Architecture §9.1 decision, then a contract description update.
ADR 0014 is still *proposed*	Commit authorization and the post-ban Remove Commit workflow are undecided.	Accept the ADR before any control-message endpoint or schema work.
`direct` conversations are unenforceable in isolation	Block, friendship, and privacy projections do not exist yet; enforcement would fail closed against empty tables.	Social-integration slice owns those three tables; the contract already defines the `403`.
Timestamps are `timestamp without time zone`	Ecto's `:utc_datetime_usec` vs the document's `timestamptz` convention — pre-existing, repo-wide.	Reconcile in the document or a migration decision; not changed in this slice.
`sender_device_id` is NOT NULL	System and control messages may eventually need a sender-less row.	Resolve when the control-message slice lands.

11 References

`contracts/openapi/chat-api.yaml`	The Chat REST contract — source of truth for every path, schema, and status code in this document.
`contracts/events/chat/*.v1.schema.json`	Event payload schemas for the four chat events.
`contracts/events/common/event-envelope.schema.json`	Shared envelope every event must satisfy.
`services/chat-service/priv/repo/migrations/20261106000000{1..7}_*.exs`	The seven neutral `chat_db` migrations.
`docs/architecture/architecture-and-schema.md`	Schema (§3), contract rules (§5), event catalog (§7), enum audit (§8).
`docs/decisions/0007-e2ee-protocol-selection.md`	Accepted: MLS, Chat as Delivery Service, opaque envelopes only.
`docs/decisions/0008-broker-transport-and-delivery-contract.md`	Accepted: HTTP claim-based pull, at-least-once, 256 KiB limit.
`docs/decisions/0014-community-commit-authorization.md`	Proposed: commit authorization and post-block removal — not encoded here.
`contracts/VERSIONING.md`	Additive vs breaking rules for REST and events.
`docs/changelog/2026-11-06-chat-contract-slice-one.md`	Change record for this slice.

How to read this document

This PDF is a generated summary. When it disagrees with `contracts/openapi/chat-api.yaml`, the architecture document, or an ADR, the source file wins and this document must be regenerated.